
```
<script type="text/javascript">
function fibonacci(nthNumber) {
    document.write('fibonacci(' + nthNumber + ') called.<br />');
    if (nthNumber === 1 || nthNumber === 2) { ❶
        // BASE CASE
        document.write('Call to fibonacci(' + nthNumber + ') returning 1.<br /
>');
        return 1;
    }
    else {
        // RECURSIVE CASE
        document.write('Calling fibonacci(' + (nthNumber - 1) + ') and fibonacci
(' + (nthNumber - 2) + ').<br />');
        let result = fibonacci(nthNumber - 1) + fibonacci(nthNumber - 2);
        document.write('Call to fibonacci(' + nthNumber + ') returning ' + result
+ '<br />');
        return result;
    }
}

document.write(fibonacci(10) + '<br />');
</script>
```

When you run this code to calculate the 10th Fibonacci number, the output looks like this:

```
fibonacci(10) called.
Calling fibonacci(9) and fibonacci(8).
fibonacci(9) called.
Calling fibonacci(8) and fibonacci(7).
fibonacci(8) called.
Calling fibonacci(7) and fibonacci(6).
fibonacci(7) called.
--snip--
Call to fibonacci(6) returning 8.
Call to fibonacci(8) returning 21.
Call to fibonacci(10) returning 55.
55
```

Much of the code is for displaying this output, but the `fibonacci()` function itself is simple. The base case—the circumstances where recursive calls are no longer made—occurs when `nthNumber` is 1 or 2 ❶. In this case, the function